White Team Documentation

CDC Team 9: Team 5

Table of Contents

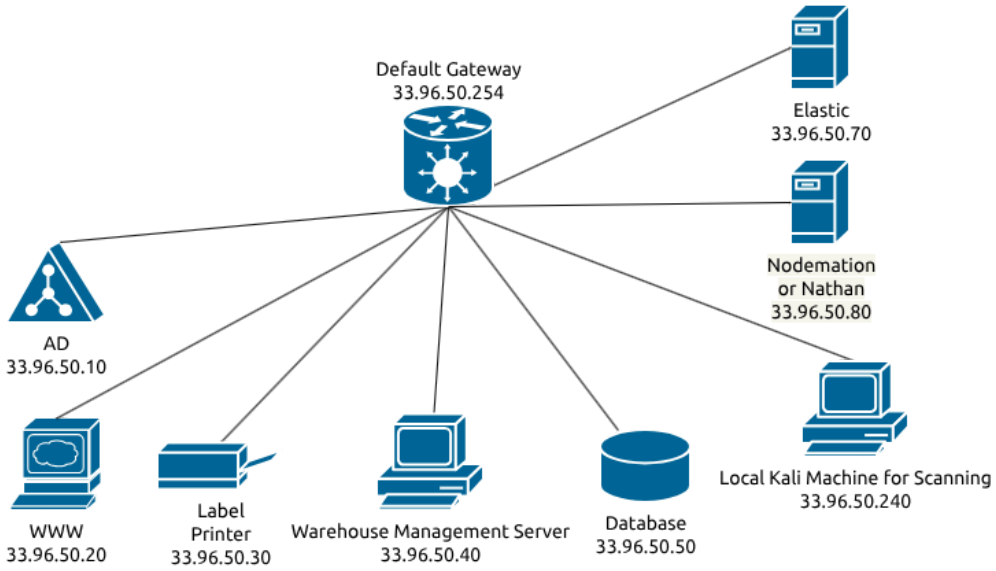
Table of Contents 1
Network Topology 2
Server Table 2
Network Diagram 2
Servers 2
Overall 2
AD 3
WWW 4
LABEL PRINTER 5
WMS 5
DATABASE 6
ELASTIC 6
Nathan 8
SCAN (aka Kali) 10

Network Topology

Server Table

Server Name	IP Address	Open Ports
ad.team9.isucdc.com	33.96.50.10	389, 3389
www.team9.isucdc.com	33.96.50.20	22, 80
lp.team9.isucdc.com	33.96.50.30	445, 3389
wms.team9.isucdc.com	33.96.50.40	22, 8080
db.team9.isucdc.com	33.96.50.50	22, 3306
elastic.team9.isucdc.com	33.96.50.70	8220, 9200
n8n.team9.isucdc.com	33.96.50.80	5678
scan.team9.isucdc.com	33.96.50.240	-NONE-

Network Diagram



Servers

Overall

Passwords: We updated the default passwords for all servers and the database. This prevents the password from being guessed as "`cdc`" or similar. The database has the two registered website admins saved in the users table. All user passwords are hashed using the `scrypt` password hashing algorithm using the `hashlib` Python standard library.

Updates: We ran updates on all machines. Windows Update was disabled on both Windows machines, mainly by disabling the `WindowsUpdate` service. This was fixed using a GPO to enforce automatic service startup. Ubuntu 16.04 was EOL and required attaching Ubuntu Pro to ensure an updated kernel and packages.

Firewalling: All machines employ a local firewall policy, using a whitelisting principle of least privilege strategy, only allowing the necessary ports for the running services (See open ports on the network diagram).

SSH: All Linux boxes use Secure Shell for remote management. SSH is restricted to only users of the AD group `IT Administrators` and does not allow `root` authentication, X11/TCP port forwarding. All servers are protected with `fail2ban` to protect against bruteforce attacks.

SMB Configuration: We disabled unencrypted SMBv1 usage network-wide, opting to enforce the encrypted SMBv2/SMBv3 protocol for network shares. This eliminates two major exploits for the Windows server versions (EternalBlue and ZeroLogon).

Backdoor Users: There were multiple identified users on all machines that were either unnecessary or existed for the express purpose of providing a means for backdoor login. Below is a table detailing all found backdoor users.

USERNAME	DETERMINED PURPOSE	LOCATION
scrat	Privileged user with insecure password for backdoor.	wms.team9.isucdc.com, ad.team9.isucdc.com, db.team9.isucdc.com
sid.john.leguizamo	Unprivileged user with insecure password for backdoor.	www.team9.isucdc.com
john.leguizamo	Privileged user with `Domain Admin` and `IT Administrator` group membership.	ad.team9.isucdc.com
manny	Unprivileged user with insecure password for backdoor.	db.team9.isucdc.com
richard	Unprivileged user with insecure password for backdoor.	db.team9.isucdc.com

AD

Undocumented Users: We found an Active Directory user named `scrat`. Since we found no function for this user and it was not in the scenario documentation, and also from its suspicious name (relating to the movie Ice Age), we decided to delete this user.

GPO Configuration: Using group policy, we enforce the following configuration:

GPO NAME	CONFIGURATION
RDP user restriction	Prevents users other than those in the `IT Administrators` security group from accessing Windows machines via RDP
Windows Defender enforcement	Enforces Windows security services(Windows Defender, Windows Firewall, and Windows time services)
Windows Update enforcement	Enforces Windows update policy

Damaged Windows Defender: The Windows Defender toolkit was destructively modified. We were able to repair it using Windows Update, DISM, and `sfc`. I was able to repair the image and restore Windows Defender functionality via the Windows Defender enforcement GPO.

WWW

Vulnerabilities: We identified and removed the following vulnerabilities

WHAT	WHERE	HOW WE FIXED
Privilege escalation through admin point abuse	The auth function will give a user admin rights if they have 200 or more loyalty points.	We removed the offending code.
SQL Injection	The DB middleware was using f-strings instead of the `MySQLdb` parameterization. capabilities.	Switched all SQL statements to use `MySQLdb` SQL statement parameterization. to prevent injection.
Disabled SELinux	SELinux was found to be unenforced with a command in `/home/cdc/.bashrc` disabling it on login.	We removed the offending command and set SELinux to `Enforcing`. Furthermore, we performed an FS relabel (ran `touch /.autorelabel`) for good measure.
Unnecessary services	Systemd services (such as `Libvirt` and `wpa_supplicant`).	We stopped, disabled, and masked all unnecessary Systemd services.
Globally readable passwords	The `/etc/shadow` was globally readable with a command at `/home/cdc/.bashrc` setting the file permissions to `0777` at login.	We removed the offending command.
Out of date Python	Python was running at `3.11`, resulting in the usage of insecure PIP packages being used.	We fetched, compiled, and manually installed into `/opt` the latest version of Python to ensure the latest Python package versions.

Website Stack Hardening: The website originally ran with the development Flask WSGI server as the `root` user and group. We have hardened the Systemd service file to remove unnecessary permissions (Unneeded network socket privileges, write privileges on system directories, etc). We changed the WSGI server to Gunicorn. This allowed us to remove the requirement of running the server as `root` by using a Systemd socket activator and forward the connections via a UNIX socket. We changed the Flask secret key to a 128-byte cryptographically-random key to prevent decryption and manipulation of session data.

LABEL PRINTER

SMB Share Settings: We configured SMB permissions to limit access to only groups with applicable usage, preventing access to all other parties.

Firewall Backdoor: There was a backdoor firewall configuration `hackr` allowing all programs to communicate on `1337/ALL`. This firewall rule was removed.

WMS

Vulnerabilities: We identified and removed the following vulnerabilities

WHAT	WHERE	HOW WE FIXED
Name Switching Service Backdoor	There is an additional directive in `/etc/nsswitch.conf` allowing password login using a NSS provider `backdoor`.	Removed the offending directive in configuration file.
Inetd Service	There is an `inet` service running in the background providing Telnet access.	Stopped, disabled, and masked the offending service. Removed `inetutils-inetd`, `inetutils-telnetd`, and `inetutils-telnet`
FTP Service	There was a running SFTP server running in the background.	Stopped, disabled, and masked the offending service. Removed `vsftpd`.
Netcat Backdoor	There was a rootful shell exposed over 1521/TCP with no authentication.	Killed socat forwarder process and removed service file `/etc/systemd/system/sid.service`.
Rogue DNS Server	There is a standalone DNS resolver running on the server. This will conflict with our static DNS settings and cause issues with authentication against AD.	Stopped, disabled, and masked the offending service. Removed `named`.

CIFS Mount Settings: The original mount line used SMBv1 with default globally-readable plaintext credentials, which is incompatible with our network configuration and password policy. Instead, we created a `root`-owned file `/etc/win-credentials` to store Windows credentials and mounted the network share under the user `wms`, eliminating the need to run the WMS server as `root` and allowing us to further sandbox the WMS service using Systemd configuration.

DATABASE

Vulnerabilities: We identified and removed the following vulnerabilities

WHAT	WHERE	HOW WE FIXED
Name Switching Service Backdoor	There is an additional directive in `/etc/nsswitch.conf` allowing password login using a NSS provider `backdoor`.	Removed the offending directive in the configuration file.

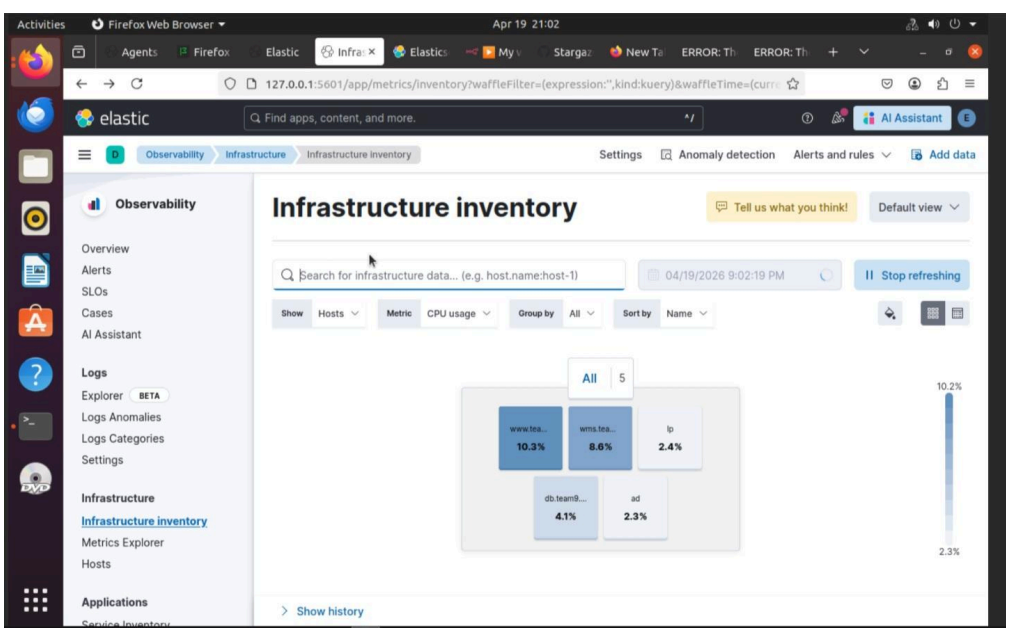
Power Delegation: All warehouse workers had MySQL users created for them so that they can modify database entries to do things like update inventory count. The permissions have been fine-tuned to enforce the principle of least privilege.

Fail2Ban: To protect MySQL from bruteforce attempts, we extended the existing Fail2Ban installation to also protect the MySQL network socket with the same parameters as SSH.

ELASTIC

What it is: Elastic is a centralized logging and monitoring platform used to collect, store, and search logs and system metrics from servers across the network in one place.

Deployment: Installed and configured Elastic and Kibana on `33.96.50.70` as the central logging server, and connected the other machines using Elastic Agents.



Logging: Configured Elastic Agents to collect Linux logs from `/var/log/*.log`, and on Windows machines to collect Security and System event logs through the Windows Event Log system, along with CPU, memory, and network metrics.

Fail2Ban: Fail2Ban was configured on the Elastic server to ban an IP after 5 failed login attempts for 15 minutes.

```
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/lib/systemd/system/fail2ban.service; enabled; vendor preset: enabled)
     Active: active (running) since Sun 2026-04-19 16:34:33 CDT; 25min ago
       Docs: man:fail2ban(1)
   Main PID: 19781 (f2b/server)
      Tasks: 5 (limit: 18617)
     Memory: 15.0M
     CGroup: /system.slice/fail2ban.service
             └─19781 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Apr 19 16:34:33 elastic systemd[1]: Starting Fail2Ban Service...
Apr 19 16:34:33 elastic systemd[1]: Started Fail2Ban Service.
Apr 19 16:34:33 elastic fail2ban-server[19781]: Server ready
cdc@elastic:~$
cdc@elastic:~$
```

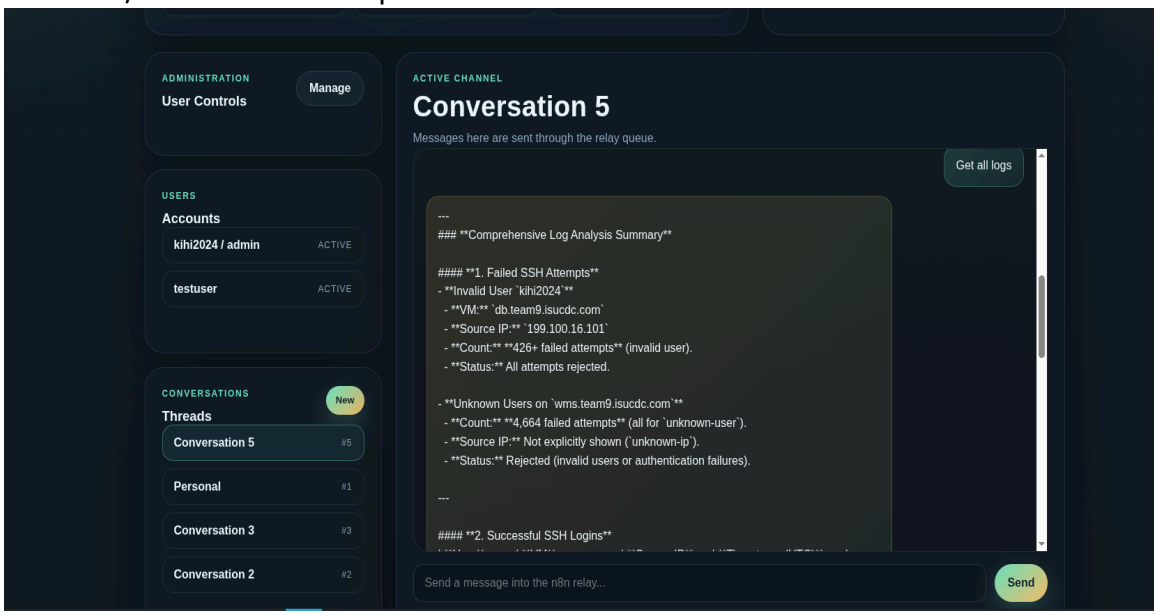
Nathan

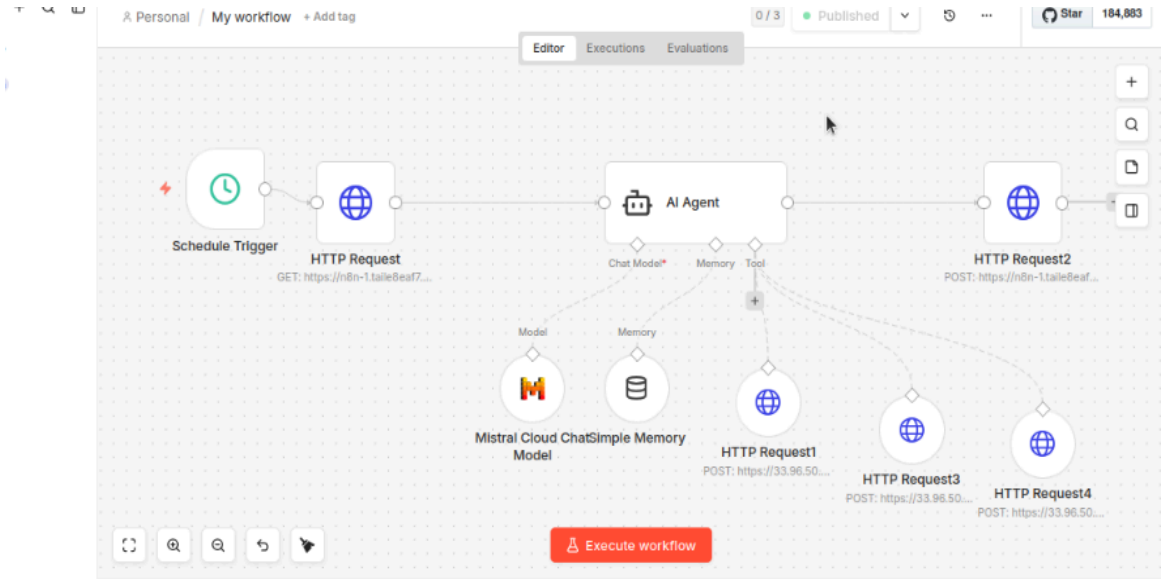
What it is: n8n is a workflow automation platform used to pull logs from Elastic, pass them through AI, and send the results back to the team.

Deployment: Deployed as the automation layer used for this year's log analysis and reporting workflow.

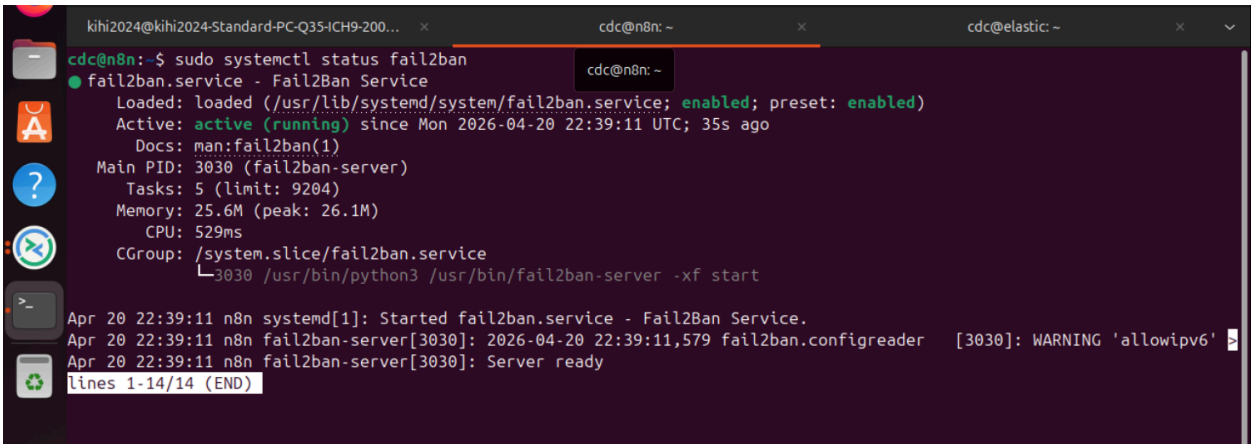
Nathan: We like to refer to this box as *Nathan* because "Nathan" starts and ends with "N", which "n8n" implies. Nathan is much easier to say than en-eight-en. We do realize that unlike the title "n8n" implies, "Nathan" is not 10 characters long. We choose to ignore this fact.

Configuration: Configured to query Elastic for log data, feed the returned results into the AI workflow, and return the response in a usable format.





Fail2Ban: Fail2Ban was configured on the n8n server to ban an IP after 5 failed login attempts for 15 minutes.



SCAN (aka Kali)

What it is: We decided to set up a Kali Linux machine to perform scanning and similar administrative actions. It does not provide any mechanical function to the network. It exists as an entry point to the network to run long scans with Nmap and Netexec (formerly known as CrackMapExec). It is not joined to AD, and cannot be remotely accessed via SSH.